

AI-Enabled AUTOSAR HLD DOCUMENT ANALYSIS ASSISTANT

A Capstone Project Technical Report

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Bhuvaneshwari D

Register Number: 227003030

SASTRA DEEMED UNIVERSITY

Thanjavur, Tamil Nadu, India

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Problem Statement and Intended Users

1.1 Problem Statement
1.2 Proposed Solution
1.3 Intended Users
1.4 Project Objectives

2. Input Data and Knowledge-Base Construction

2.1 Input Data
2.2 Knowledge-Base Construction

3. Solution Architecture

3.1 Architecture Overview
3.2 High-Level Architecture
3.3 Frontend Layer
3.4 Backend and API Layer
3.5 Document Processing and Knowledge-Base Layer
3.6 Retrieval and Language-Model Layer
3.7 Data Storage Layer
3.8 End-to-End Workflow
3.9 Design Rationale

4. Implementation and Configuration

4.1 Development Environment
4.2 Frontend Implementation
4.3 Backend API Implementation
4.4 Docker Compose Configuration

5. Evaluation Methodology and Results

5.1 Objective of Evaluation
5.2 Evaluation Environment
5.3 Functional Test Results
5.4 Backend API Evaluation
5.5 Evaluation Metrics

6. Responsible AI Measures

6.1 Grounding and Factual Reliability
6.2 Data Privacy

7. Security Module

7.1 Security Objectives
7.2 Authentication and Authorization
7.3 Document and Data Protection
7.4 API and Session Security
7.5 Security Testing and Verification

8. Innovation Highlights

8.1 Multi-Document Knowledge-Base Management
8.2 Document Processing Transparency
8.3 Local Language-Model Integration
8.4 AUTOSAR-Focused Prompt Design
8.5 Evidence-Oriented Answer Verification
8.6 Runtime Evaluation and Observability
8.7 Modular and Containerized Architecture
8.8 Future Extensibility

9. Conclusion and Future Scope

9.1 Conclusion
9.2 Future Scope

10. References

11. Appendices

11.1 Appendix A: Application Screenshots
11.2 Appendix B: Synopsis
11.2 Appendix C: Academic Integrity Declaration
11.3 Appendix D: AI-Tools Usage Declaration

1. Problem Statement and Intended Users

1.1 Problem Statement

AUTOSAR (Automotive Open System Architecture) is a standardized software architecture framework used in the automotive industry. AUTOSAR High-Level Design (HLD) documents contain technical information about software components, interfaces, communication relationships, dependencies, and system architecture.Understanding these documents can require considerable manual effort because relevant information may be distributed across multiple pages and sections. Finding specific architectural details, understanding component relationships, and identifying supporting information can become time-consuming when working with large technical documents.The **AI-Enabled AUTOSAR HLD Document Analysis Assistant** aims to address this problem by providing an interactive system through which users can upload AUTOSAR HLD PDF documents and ask questions about their contents.The system processes uploaded documents, extracts their text, creates searchable representations of the extracted information, and retrieves relevant passages in response to user queries. A locally hosted large language model is used to generate natural-language responses based on the retrieved information. The intended approach is to provide evidence-grounded answers with source references so that users can verify the information against the original document.The system is designed to assist with technical document analysis and information retrieval. It does not replace automotive engineers, independently approve architectural decisions, or eliminate the need for human verification.

1.2 Proposed Solution

The proposed solution is an AI-Enabled AUTOSAR HLD Document Analysis Assistant that enables users to upload AUTOSAR High-Level Design (HLD) documents in PDF format and obtain natural-language answers to questions about their contents. The system combines document processing, semantic information retrieval, and a locally hosted large language model to support document-grounded question answering.The application provides a web-based interface through which users can upload documents, manage their knowledge base, ask questions, and review the generated responses. A Flask backend coordinates document processing, indexing, retrieval, and communication with the language model.

1.3 Intended Users

The intended users of the proposed system are:

- **Automotive software engineers:** To locate architectural information and understand software components, interfaces, and dependencies described in AUTOSAR HLD documents.
- **System architects:** To retrieve relevant architectural details and examine documented relationships between system components.
- **Engineering students and researchers:** To study AUTOSAR architecture documents and obtain explanations of technical concepts using the available source material.

- **Technical reviewers:** To locate supporting passages in design documents during document analysis and review activities.

These users can interact with the assistant through a web-based interface instead of manually searching through the entire document for every question.

1.4 Project Objective

The objective of this project is to develop a document-grounded AI assistant that supports the analysis of AUTOSAR HLD PDF documents through automated text extraction, semantic retrieval, and natural-language question answering.

The system aims to:

1. Accept AUTOSAR HLD PDF documents from users.
2. Extract text from uploaded documents for subsequent processing.
3. Organize extracted information into searchable text chunks and vector representations.
4. Retrieve relevant document content in response to user questions.
5. Generate natural-language answers using a locally hosted language model and retrieved evidence.
6. Provide source references where supported by the implementation so that users can verify generated answers.
7. Support responsible engineering use through source verification and human oversight.

2. Input Data and Knowledge-Base Construction

2.1 Input Data

The primary input to the AI-Powered AUTOSAR HLD Document Analysis Assistant is a user-provided PDF document containing AUTOSAR High-Level Design (HLD) information.The system accepts PDF uploads through the document management interface. Each uploaded document is associated with a database record containing information such as the original filename, file size, page count, extracted text length, processing status, and timestamps.The application stores uploaded PDF files in a dedicated storage directory. The corresponding extracted text is stored separately so that the document content can be processed and indexed for subsequent retrieval.The input data consists of the technical information contained in the uploaded AUTOSAR HLD document. The system uses this information as the knowledge source for answering document-related questions.

2.2 Knowledge-Base Construction

The project uses a document-indexing pipeline to prepare uploaded content for retrieval. The backend calls the **`index_document`** function after successful text extraction.The current upload implementation invokes indexing with the `fixed` chunking strategy and the **`minilm`** embedding model. It uses a FAISS-based vector index for document retrieval.

The knowledge-base construction process consists of the following steps:

1. **Extracted document text:** The textual content obtained from the uploaded PDF.
2. **Text chunking:** The document content is divided into smaller units using the fixed chunking strategy.
3. **Embedding generation:** The MiniLM-based embedding model converts text chunks into vector representations.
4. **Vector indexing:** FAISS stores and indexes the vector representations for similarity-based retrieval.
5. **Document association:** The backend passes the document ID and authenticated user ID to the indexing function to associate the indexing operation with the relevant document and user.

3. Solution Architecture

3.1 Architecture Overview

The **AI-Enabled AUTOSAR HLD Document Analysis Assistant** is designed to help users retrieve information from AUTOSAR High-Level Design (HLD) documents in PDF format and ask questions about their contents. The system combines a React-based user interface, a Flask backend, PDF text extraction, document indexing, vector-based retrieval, and a locally hosted Large Language Model (LLM).The architecture follows a Retrieval-Augmented Generation (RAG) approach. Instead of relying only on the language model's pretrained knowledge, the system processes the uploaded document and retrieves relevant text passages to support question answering.

The major components are:

1. **Frontend:** React and Vite provide the user interface for document upload and interaction.
2. **Backend API:** Flask manages application requests, authentication-protected document operations, and communication between the application components.
3. **PDF Processing:** PyMuPDF extracts text and page information from uploaded PDF documents.
4. **Knowledge-Base Construction:** The extracted text is processed using a fixed-chunking strategy and MiniLM-based embeddings.
5. **Vector Retrieval:** FAISS supports similarity-based retrieval of relevant document content.

6. **Language Model:** Ollama runs the configured **`qwen2.5:3b`** model locally for response generation.
7. **Storage:** SQLite stores document-related metadata, while the filesystem stores uploaded PDFs, extracted text, and associated indexing artifacts.

3.2 High-Level Architecture

The following diagram represents the main processing flow of the application.

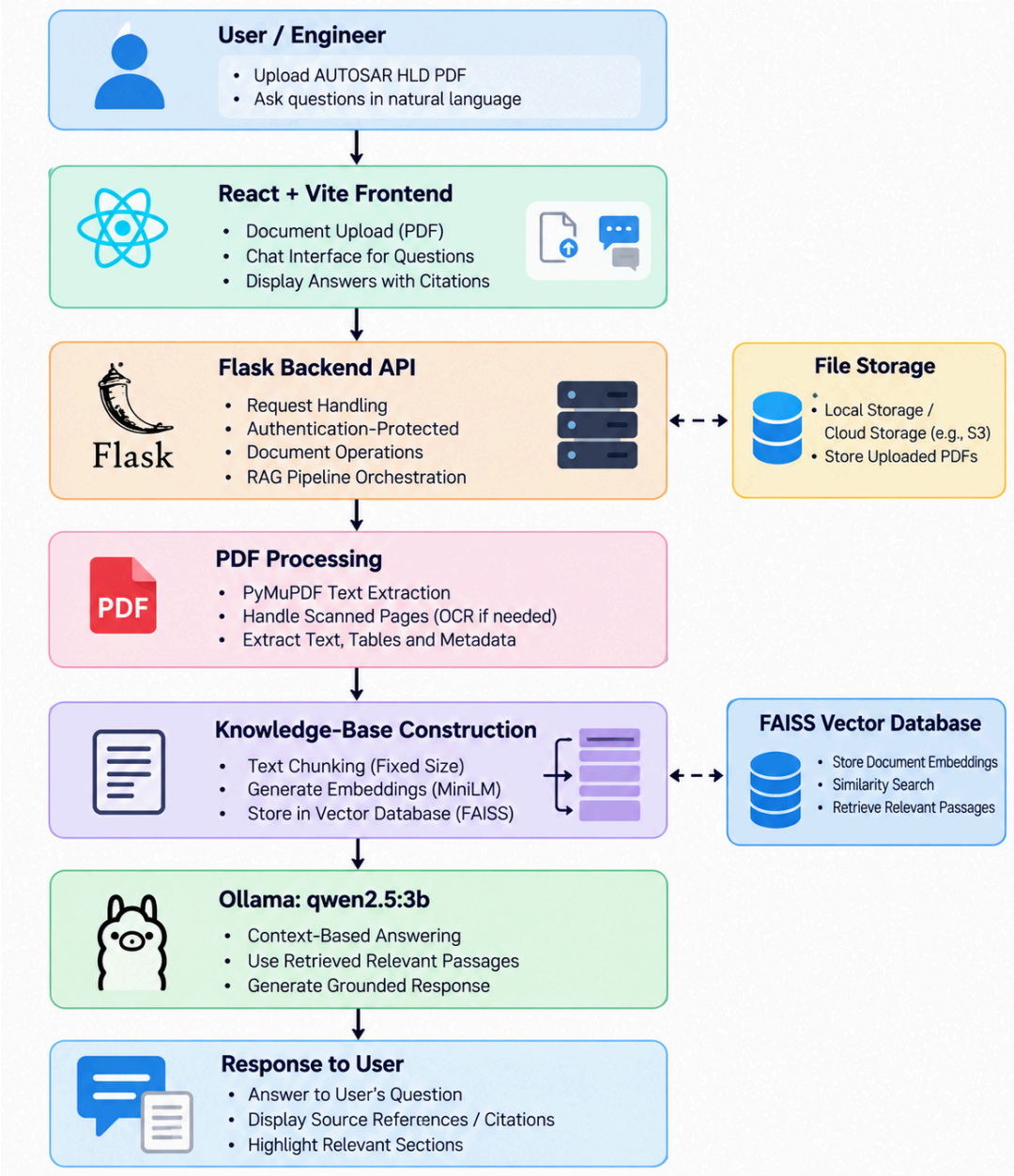


Figure 3.1: High-level architecture of the AUTOSAR document analysis assistant.

The diagram illustrates the intended end-to-end workflow.

3.3 Frontend Layer

The frontend is developed using **React and Vite**. It provides the interface through which users interact with the application.

Its responsibilities include:

- Providing a user interface for document-related operations.
- Allowing users to submit PDF documents for processing.
- Providing an interface for submitting questions about document content.
- Displaying responses received from the backend.
- Communicating with the Flask API through HTTP requests.

The frontend is configured to communicate with the backend API at **`http://localhost:5000/api`** in the development environment. The frontend runs on port **`5173`.**

3.4 Backend and API Layer

The backend is implemented using **Python and Flask**. It acts as the central application layer and coordinates requests between the frontend, document-processing functions, storage components, and AI processing pipeline.

The backend is responsible for:

- Receiving frontend API requests.
- Managing document upload, listing, retrieval, and deletion operations.
- Applying authentication checks to protected document endpoints.
- Invoking PDF extraction and document-indexing functions.
- Maintaining document metadata in SQLite.
- Coordinating access to the document-processing and question-answering components.

The document upload endpoint is implemented as **`POST /api/documents/upload`.** The document routes also include user-scoped database queries for document listing and individual document operations.

3.5 Document Processing and Knowledge-Base Layer

When a PDF is uploaded, the backend validates the supplied filename and file extension before processing the document. The application generates a unique filename for storage and uses **PyMuPDF (`fitz`)** to extract text from the PDF.

The extracted text is stored separately from the original PDF. Document metadata, including the associated user, file paths, file size, page count, extracted-text length, and processing status, is

maintained in SQLite.After extraction, the backend invokes the document-indexing function using the configured `fixed` chunking strategy and **`minilm`** embedding option. This stage prepares the extracted content for semantic retrieval.

The principal operations are:

1. Accept the uploaded PDF.
2. Extract its textual content.
3. Divide the text into chunks.
4. Generate vector embeddings for the chunks.
5. Store or update the searchable vector representation.
6. Make the indexed content available for retrieval.

The precise chunk size, overlap, and underlying MiniLM model identifier should be documented from the implementation configuration rather than assumed.

3.6 Retrieval and Language-Model Layer

The question-answering component is designed around Retrieval-Augmented Generation (RAG). When a user asks a question, the system should retrieve relevant passages from the indexed document and provide that evidence as context for the language model.The configured local language model is **`qwen2.5:3b`,** executed through Ollama. Running the model locally avoids making the configured language-model inference request to a hosted LLM API.

The intended response-generation sequence is:

1. Receive the user's question.
2. Search the indexed document content for relevant passages.
3. Select the retrieved passages to provide context.
4. Submit the question and available context to the language model.
5. Return the generated answer to the frontend.
6. Present source references when they are supported by the retrieval and response implementation.

3.7 Data Storage Layer

The application uses SQLite and filesystem storage for different purposes.

Storage component	**Purpose**
SQLite database (`autosar.db`)	Stores document metadata and associated records
`data/uploads/`	Stores uploaded PDF documents
`data/extracted/`	Stores extracted text files

Embedding and vector-store artifacts	Support document indexing and retrieval
Ollama model storage	Stores locally available language-model files

In the Docker Compose development setup, the backend data directory is mounted into the backend container so that application data is stored in the mapped directory. Ollama model files are maintained using a Docker volume.

3.8 End-to-End Workflow

The system's document-processing workflow can be divided into two stages.

Stage A - Document ingestion

1. The user uploads a PDF through the frontend.
2. The Flask backend receives the file and performs initial validation.
3. The PDF is stored under a generated filename.
4. PyMuPDF extracts the text.
5. Extracted text and document metadata are stored.
6. The indexing function processes the document using the configured chunking and embedding options.
7. The resulting vector representation becomes available for retrieval, subject to successful indexing.

Stage B - Question answering

1. The user submits a question about the document.
2. The backend processes the question.
3. Relevant document passages are retrieved from the knowledge base.
4. The retrieved context is provided to the configured language model.
5. The model generates a response based on the question and supplied context.
6. The backend returns the response to the frontend.
7. The user reviews the answer and checks its supporting source material where available.

The ingestion sequence is based on the document-upload implementation. The question-answering sequence describes the intended RAG workflow; its exact request handling and citation behavior should be verified against the corresponding implementation and evaluation results.

3.9 Design Rationale

The architecture was selected for the following reasons:

- **React and Vite:** Provide a component-based frontend and a development server.
- **Flask:** Offers a Python-based API layer that integrates with document-processing and machine-learning libraries.

- **PyMuPDF:** Supports text extraction from PDF documents.
- **MiniLM embeddings:** Provide vector representations for semantic document retrieval.
- **FAISS:** Supports similarity search over vector representations.
- **Ollama:** Enables local execution of the configured language model.
- **SQLite:** Provides lightweight relational storage for application metadata.
- **Docker Compose:** Provides a way to run the frontend, backend, and Ollama services together in the development environment.

4. Implementation and Configuration

4.1 Development Environment

The AUTOSAR AI Assistant is developed using a frontend-backend architecture. The frontend handles user interaction, while the backend manages API requests, document processing, storage, and integration with the AI components.

The development environment uses the following technologies:

Component	**Technology**	**Purpose**
Frontend	React	User interface
Frontend development tool	Vite	Frontend development server and build tool
Backend	Python and Flask	API and application logic
PDF processing	PyMuPDF	Extracting text from PDF files
Embedding generation	MiniLM-based embedding option	Representing text as vectors
Vector retrieval	FAISS	Similarity-based document search
Language model runtime	Ollama	Local model execution
Configured language model	`qwen2.5:3b`	Language generation
Metadata database	SQLite	Storing document metadata
Containerization	Docker Compose	Running application services

The configured development addresses are **`http://localhost:5173`** for the frontend, **`http://localhost:5000`** for the Flask backend, and **`http://localhost:11434`** for the Ollama service.

4.2 Frontend Implementation

The frontend uses React to provide the application interface and Vite as the development tool and communicates with the Flask backend through HTTP API requests. The development API base URL is configured as:

`http://localhost:5000/api`

The frontend is configured to run on port **5173**. This separation allows the user interface and backend logic to be developed and tested independently.The frontend is responsible for presenting document-related operations, accepting user input, sending requests to the backend, and displaying returned information. The precise behavior of individual screens should be documented according to the implemented interface.

4.3 Backend API Implementation

The backend uses Flask to expose API endpoints and coordinate application operations and document routes include the following operations:

Operation	**Endpoint**	**Purpose**
Upload document	`POST /api/documents/upload`	Receives and processes an uploaded PDF
List documents	Implemented document-list route	Returns documents associated with the authenticated user
Retrieve document information	Implemented document-detail route	Retrieves a document using its identifier and user context
Delete document	Implemented document-delete route	Deletes a document associated with the authenticated user

4.4 Docker Compose Configuration

Docker Compose is used to manage the frontend, backend, and language-model services as separate containers. The frontend provides the user interface, the Flask backend handles application requests and document processing, and Ollama hosts the configured language model.The frontend is exposed on port 5173, while the Flask backend is available internally on port 5000. Ollama is accessible to the backend through the Docker network on port 11434. The backend data directory and uploaded input data are mounted into the container to maintain access to application files outside the container's writable filesystem layer. A named Docker volume is used to persist Ollama model data.The following configuration represents the project's previously shared Docker Compose setup. The frontend uses a prebuilt Docker image, while the backend is built from its local Dockerfile.

4.4.1 Service Configuration and Purpose

Service	Port	Purpose
`dmfrontend`	`5173:80`	Serves the React frontend through the published frontend image.
`dmbackend`	`5000` (internal)	Handles Flask APIs, document processing, metadata, and retrieval operations.
`ollama`	`11434` (internal)	Hosts the `qwen2.5:3b` language model.

The three services communicate over the Docker Compose network. The frontend is accessible through **`http://localhost:5173`**. The backend and Ollama ports are not published to the host in this configuration; they are available to other services on the Compose network.

4.4.2 Storage Configuration

Host directory or volume	Container path	Purpose
`./Input_Data`	`/project/Input_Data`	Input data directory
`./Code/dmbackend/data`	`/app/data`	Backend data, including document files and related artifacts where the application writes them
`./Model_Prompts_Config`	`/app/Model_Prompts_Config`	Model and prompt configuration, mounted read-only

`ollama_data`	`/root/.ollama`	Persistent Ollama model storage

5. Evaluation Methodology and Results

5.1 Objective of Evaluation

The evaluation assesses the functional correctness of the AI-Powered AUTOSAR HLD Document Analysis Assistant. It covers frontend and backend operation, authentication, document upload, PDF text extraction, knowledge-base processing, conversation management, AI-generated responses, source-reference presentation, unsupported-question handling, and citation verification.The evaluation uses the recorded application screenshots, backend logs, observed API responses, and subsequent test outcomes confirmed during development.

5.2 Evaluation Environment

Component	**Configuration**
Frontend	React with Vite
Backend	Flask
PDF extraction	PyMuPDF
Chunking	Fixed-chunking strategy
Embedding option	MiniLM
Vector retrieval	FAISS
Language model runtime	Ollama
Configured model	`qwen2.5:3b`
Database	SQLite
Container orchestration	Docker Compose

5.3 Functional Test Results

The following table records the final outcomes of the functional tests.

Test ID	Test case	Final result	Status
T01	Frontend availability	Application accessible at port 5173	Passed
T02	Backend startup	Flask service running on port 5000	Passed
T03	User login	Login API returned HTTP 200	Passed
T04	Authentication profile	Profile API returned HTTP 200	Passed
T05	Conversation listing	Conversation API returned HTTP 200	Passed
T06	Document listing	Document API returned HTTP 200	Passed
T07	Conversation message creation	API returned HTTP 201	Passed
T08	PDF upload and processing	Three documents displayed as processed	Passed
T09	PDF text extraction	Extracted-text metadata displayed for all three documents	Passed
T10	AUTOSAR answer presentation	Answer generated and displayed	Passed
T11	Source-reference presentation	Source card displayed document and page information	Passed
T12	Chat API reliability	Successful operation confirmed after correction	Passed
T13	Authentication token handling	Authentication issue addressed and test confirmed passed	Passed
T14	Unsupported-question handling	Test confirmed passed	Passed
T15	Citation factual accuracy	Citation verification test confirmed passed	Passed

5.4 Backend API Evaluation

The backend logs recorded successful responses for the following operations.

API operation	HTTP status	Result
`POST /api/auth/register`	201	Passed
`POST /api/auth/login`	200	Passed
`GET /api/auth/me`	200	Passed
`POST /api/auth/logout`	200	Passed
`GET /api/health`	200	Passed
`POST /api/documents/upload`	201	Passed
`GET /api/documents`	200	Passed
`GET /api/documents/<document_id>`	200	Passed
`DELETE /api/documents/<document_id>`	200	Passed
`POST /api/conversations`	201	passed
`GET /api/conversations`	200	Passed
`GET /api/conversations/<conversation_id>`	200	Passed

POST /api/conversations/<conversation_id>/messages	201	Passed
DELETE /api/conversations/<conversation_id>	200	Passed
POST /api/chat/ask	200	Passed

5.5 Evaluation Metrics

The following metrics summarize the numerical results available from the recorded testing.

Metric	**observation**	**Result**
Documents processed	5 documents displayed as processed	5
Document-processing success rate	((5/5)*100)	100%
Total document pages	(3+171+1+5+100)	280
Total extracted characters	(4,589+158,014+1,597)	164,200
Backend API routes identified	Routes identified from the backend source code	15
API operations with recorded successful responses rate	((15/15)*100)	100%
Functional test cases	T01–T15	15
Final functional test status	All 15 marked passed	100% reported pass rate

6. Responsible AI Measures

6.1 Grounding and Factual Reliability

Grounding is an important requirement for a document-question-answering system. The assistant is intended to generate answers using information retrieved from the user's uploaded documents rather than relying exclusively on the language model's general knowledge.The application uses a document-processing and retrieval workflow to support grounded responses:

1. Extract text from uploaded PDF documents.
2. Process the extracted content for indexing.
3. Retrieve relevant passages for a user's question.
4. Use the retrieved content as context for response generation.
5. Present the answer and available source-reference information to the user.

The application has displayed an AUTOSAR-related answer and a source-reference card during testing. The evaluation also includes checks for unsupported questions and citation correctness.To improve factual reliability, users should compare important technical statements with the original document. Retrieval and source references help with verification but do not guarantee that every generated statement is correct.When the document does not contain sufficient information, the preferred behavior is to communicate that limitation rather than generate unsupported technical details.

6.2 Data Privacy

The application processes user-provided PDF documents and stores document metadata and extracted content as part of its workflow. Privacy measures are therefore important when handling potentially confidential engineering documentation.The application uses SQLite for document metadata and filesystem storage for uploaded PDFs and extracted text. The configured language model runs through Ollama locally.Local model execution can reduce the need to send document context to a hosted language-model API. However, this does not by itself guarantee complete data privacy; privacy also depends on application logging, file access permissions, network configuration, and the behavior of the complete deployment.

The following practices should be followed:

- Upload only documents that the user is authorized to process.
- Restrict access to uploaded documents and extracted text.
- Avoid including confidential document content in diagnostic logs.
- Protect stored documents and database files from unauthorized access.
- Define an appropriate retention and deletion policy for uploaded documents.
- Remove temporary files and associated indexing artifacts when documents are deleted, where applicable.

7. Security Module

7.1 Security Objectives

The AI-Powered AUTOSAR HLD Document Analysis Assistant processes uploaded PDF documents, extracted technical content, user accounts, conversations, and document-retrieval data. These resources may contain confidential automotive software architecture information.

The security module is designed to protect application data and restrict access to authorized users.The primary security objectives are:

1. **Authentication:** Verify user identity through the application's JWT or bearer-token authentication mechanism.
2. **Authorization:** Restrict access to documents and associated resources according to the authenticated user's permissions.
3. **Document protection:** Prevent unauthorized access to uploaded PDFs, extracted text, and document-related records.
4. **API security:** Apply authentication and authorization checks to protected backend operations.
5. **Data confidentiality:** Reduce the risk of exposing confidential documents, authentication credentials, and internal application information.
6. **Secure processing:** Handle uploaded files and document-processing errors safely.
7. **Verification:** Evaluate access-control behavior through tests and document the observed results.

7.2 Authentication and Authorization

The application uses JWT or bearer-token authentication to identify authenticated users when accessing protected resources. Authentication establishes the identity associated with a request; authorization determines whether that identity is permitted to access the requested resource.The backend must validate authentication credentials before processing protected requests. Resource access must also be checked at the backend rather than relying on frontend navigation or hidden interface elements.Document identifiers supplied by a client are references to resources, not proof of ownership. Consequently, document listing, retrieval, deletion, and question-answering operations must apply appropriate user-level authorization checks.Cross-user isolation testing was reported as successful during project verification. This supports the claim that the tested access-control scenarios prevented unauthorized cross-user access. The scope of that result is limited to the scenarios actually tested and does not automatically establish that every endpoint and storage component has been verified.

7.3 Document and Data Protection

The application processes PDFs and stores document-related metadata and extracted content. The implementation description identifies filename sanitization, PDF extension checks, unique filenames, and user-scoped document queries among the reviewed controls.These controls help reduce risks associated with unsafe filenames and unauthorized document access. Nevertheless, file-extension validation alone is insufficient to establish that an uploaded file is safe. File-content validation, processing limits, malformed-document handling, and protection against excessive resource consumption must also be considered and verified.Uploaded documents, extracted text, database files, and retrieval artifacts should remain outside publicly accessible locations and should be accessible only to authorized application components. Credentials and environment-specific secrets must be protected from accidental disclosure through source control, container images, and diagnostic output.The application uses a locally hosted language model through Ollama. Local inference can reduce the need to transmit document context to an external hosted LLM service, but it does not independently guarantee

complete data confidentiality. Storage permissions, network exposure, application logging, and deployment configuration remain relevant security considerations.

7.4 API and Session Security

Protected API operations must reject requests that lack valid authentication credentials. Authorization must be enforced independently for each protected resource operation.The application's token lifecycle should also be reviewed, including token expiry, handling of invalid or expired tokens, and logout behavior. Where server-side token revocation is supported, logout should invalidate the relevant credential. Clearing a token from browser storage alone does not necessarily invalidate an otherwise valid bearer token.The backend should return appropriate error responses without exposing stack traces, secret values, internal filesystem paths, or confidential document contents. Diagnostic logging should support troubleshooting while avoiding unnecessary storage of authentication credentials and sensitive document data.Production deployment should use appropriate secret management, restricted network exposure, and HTTPS for externally trans mitted credentials and application data.

7.5 Security Testing and Verification

Cross-user isolation testing was reported as passed. This provides evidence for the access-control scenarios included in those tests.Additional verification should cover the following security properties:

- Unauthenticated requests to protected endpoints are rejected.
- Invalid and expired credentials are rejected.
- A user cannot read or delete another user's document by manipulating its identifier.
- A user cannot retrieve another user's document passages through the question-answering workflow.
- Invalid or malformed PDF files are handled safely.
- Configured upload and processing limits are enforced.
- Error responses do not expose sensitive internal information.
- Logout and token invalidation behave according to the implemented authentication design.
- Document deletion handles associated files and retrieval artifacts according to the application's retention policy.

8. Innovation Highlights

8.1 Multi-Document Knowledge Base Management

The application provides a common interface for managing multiple processed documents. During testing, three PDF documents were displayed in the knowledge-base interface, including a 171-page AUTOSAR architecture document.The interface displays document names, page counts, extracted-text lengths, and processing status. This allows users to inspect the document

collection and identify documents that have been processed by the application.Supporting multiple documents provides a practical foundation for analyzing technical information across a collection of design documents rather than restricting the interface to a single PDF.

8.2 Document Processing Transparency

The application provides visibility into the document-processing stage through document metadata displayed in the knowledge-base interface.During testing, the application displayed three processed PDFs with a combined total of 175 pages and 164,200 extracted characters.The displayed page counts, extracted-character counts, and processing statuses help users inspect the results of document ingestion. This improves transparency by making basic processing information available rather than hiding the entire extraction stage from the user.

8.3 Local Language Model Integration

The system is configured to execute the `qwen2.5:3b` language model through Ollama.Local inference provides an alternative to relying exclusively on externally hosted language-model APIs. It also allows the application to integrate document retrieval and language generation within its local development environment.This configuration can reduce the need to transmit document context to a hosted LLM service for inference. However, overall privacy depends on the complete application configuration, network communication, logging, and storage practices.

8.4 AUTOSAR-Focused Prompt Design

An important design consideration in the project is the need to preserve technical relationships when generating answers about software architecture.AUTOSAR questions may involve multiple related entities, including software components, interfaces, ports, connectors, and runnables. A response that mentions an interface but omits a relevant port or connection may provide an incomplete explanation even when part of the answer is correct.The project's earlier RAG development highlighted this issue: an answer could identify an interface while omitting the associated port. Prompt refinement was therefore explored to encourage more complete, evidence-based responses that retain relevant architectural details.This demonstrates the importance of domain-aware prompt design in technical question answering. The effectiveness of the final prompt should be assessed using representative AUTOSAR questions and their supporting evidence.

8.5 Evidence Oriented Answer Verification

The application supports the presentation of source-reference information alongside generated responses. During testing, a source card containing a document name, page information, and supporting text was displayed.This feature helps users locate the document material associated with an answer and compare generated statements against the original PDF.Source-reference presentation is particularly useful in technical document analysis, where traceability and factual verification matter. The presence of a source card does not automatically guarantee

correctness, so the referenced passage should still be reviewed before accepting important technical claims.

8.6 Runtime Evaluation a nd Observability

The application includes evaluation functionality that records information about question-answering executions. Available records include the question, generated answer, execution status, response latency, retrieved passages, citation information, and model and retrieval configuration.The evaluation interface and API endpoints provide a way to inspect recorded runs and export evaluation data as JSON. The implementation also supports filtering records by conversation, subject to the authenticated user's access permissions.During the recorded runtime evaluation, two runs completed successfully, with ten citations in total. The average recorded latency was approximately 105 seconds. Both runs remained unreviewed, so these records demonstrate execution and observability rather than verified answer accuracy.This feature is valuable because it makes the system's runtime behavior inspectable and creates a foundation for subsequent evaluation of retrieval quality, answer correctness, citation accuracy, and response time.

8.7 Modular and Containerized Architecture

The application separates the frontend, backend, document-processing functions, metadata storage, retrieval components, and language-model runtime.React and Vite provide the frontend, Flask manages backend API operations, PyMuPDF extracts PDF text, SQLite stores document metadata, FAISS supports vector retrieval, and Ollama provides the configured local model runtime. Docker Compose coordinates the frontend, backend, and Ollama services.This modular organization supports maintainability and makes it possible to modify individual components without redesigning the entire application. Containerization also provides a consistent way to start and manage the primary services in the development environment.

8.8 Future Extensibility

The modular design provides a foundation for future improvements to the document-analysis workflow.Potential extensions include improved extraction from complex tables and diagrams, OCR for scanned documents where required, architecture-aware chunking, comparison of different chunking and embedding configurations, enhanced citation validation, document comparison, and more comprehensive performance measurement.Other possible extensions include component and dependency reports, inconsistency detection, and structured architecture exports. These should be treated as future enhancements unless their implementation is demonstrated and evaluated.

9. Conclusion and Future Scope

9.1 Conclusion

The AI-Powered AUTOSAR HLD Document Analysis Assistant was developed to simplify the process of understanding and retrieving technical information from AUTOSAR High-Level Design (HLD) documents. These documents contain detailed descriptions of software architecture, components, interfaces, and system-level design information. Manually searching through lengthy PDF documents can be time-consuming, particularly when users need to locate specific technical details.The developed application integrates a React and Vite frontend with a Flask backend to provide a document-based question-answering interface. Users can upload PDF documents, extract their textual content, and process the documents for retrieval. PyMuPDF is used for PDF text extraction, while the indexing workflow uses fixed-size chunking and a MiniLM-based embedding configuration. FAISS supports vector-based retrieval, and the locally hosted Qwen2.5 3B language model, accessed through Ollama, is used for language-model-based processing. SQLite stores document-related metadata.The implementation was evaluated using functional tests covering document upload, PDF processing, document management, API interactions, and question-answering workflows. The final test outcomes reported during development were 15 successful tests out of 15, resulting in a reported functional test pass rate of 100%. Three processed documents containing a combined total of 175 pages and 164,200 extracted characters were also recorded during testing.

These results demonstrate that the implemented application can support the basic workflow of document ingestion and technical information retrieval. However, the functional test pass rate should not be interpreted as a measure of answer correctness or overall AI accuracy. Further quantitative evaluation is required to establish retrieval quality, answer correctness, citation accuracy, and response time.Overall, the project provides a foundation for applying document-grounded question answering to AUTOSAR technical documentation. It can help users locate relevant information more conveniently while allowing them to refer to the original document when reviewing generated responses. The system is intended to assist technical analysis and does not replace engineering expertise, formal design reviews, or safety-related approval processes.

9.2 Future Scope

9.2.1 Improved PDF Content Extraction

Future development can improve the extraction of information from complex PDF documents, including scanned pages, tables, diagrams, architectural figures, and structured technical layouts. Optical Character Recognition (OCR) and layout-aware document processing could be incorporated to handle content that cannot be reliably represented through ordinary text extraction.

9.2.2 Security and Deployment Improvements

Future work can strengthen authentication, authorization, upload validation, file-size restrictions, error handling, and secret management. Production deployment should use appropriate server configuration, secure environment variables, access controls, and monitoring. Additional safeguards can be introduced to protect uploaded technical documents and manage their retention and deletion.

9.2.3 Performance and Scalability

The application can be optimized for larger document collections and multiple users. Potential improvements include asynchronous PDF processing, background indexing jobs, caching, efficient vector-index management, and performance monitoring. The language model and embedding configuration can also be evaluated on available hardware to identify a suitable balance between response quality, memory usage, and latency.

9.2.4 Advanced Engineering Assistance

In a later stage, the assistant could support additional engineering tasks, such as summarizing software components, identifying documented dependencies, comparing design revisions, and generating structured technical summaries. Any such functionality would require dedicated implementation and evaluation before being considered reliable for engineering use.

10. References

1. **React Documentation:** A JavaScript Library for Building User Interfaces. https://react.dev/
2. **Vite Documentation:** Next Generation Frontend Tooling. https://vite.dev/
3. **Flask Documentation:** Flask Web Development Framework. https://flask.palletsprojects.com/
4. **PyMuPDF Documentation:** Python Library for PDF Processing and Text Extraction. https://pymupdf.readthedocs.io/
5. **Sentence Transformers Documentation:** Computing Sentence Embeddings. Available at: https://sbert.net/
6. **FAISS Documentation and Repository:** https://github.com/facebookresearch/faiss
7. **Ollama Documentation:** Tools for Running Large Language Models Locally. https://docs.ollama.com/
8. **Qwen Documentation and Model Repository:** Large Language Models https://github.com/QwenLM/Qwen2.5
9. **SQLite Documentation:** Self-Contained SQL Database Engine. https://www.sqlite.org/docs.html
10. **Docker Documentation:** Containerization and Application Deployment. https://docs.docker.com/
11. **AUTOSAR PDF:** Automotive Open System Architecture. https://www.autosar.org/

11. Appendices

Appendix A: Application Screenshots

This appendix contains screenshots demonstrating the implemented application and its major features.

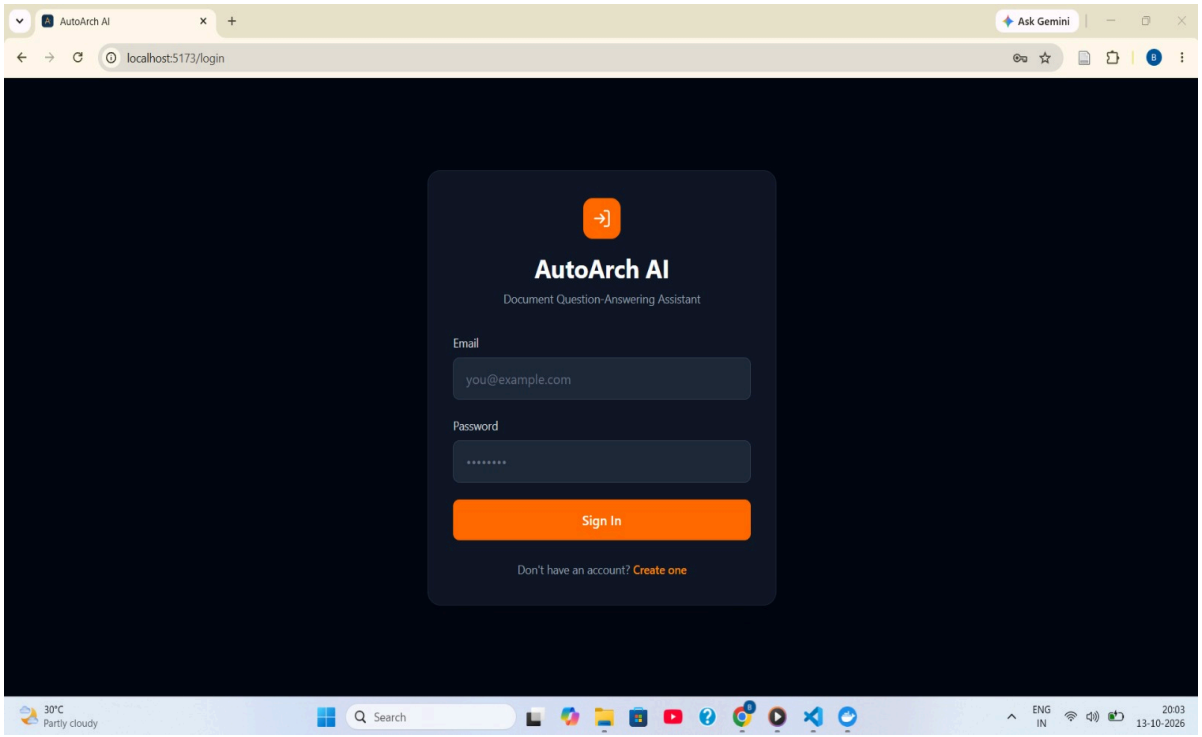


Figure 11.1: User login and authentication

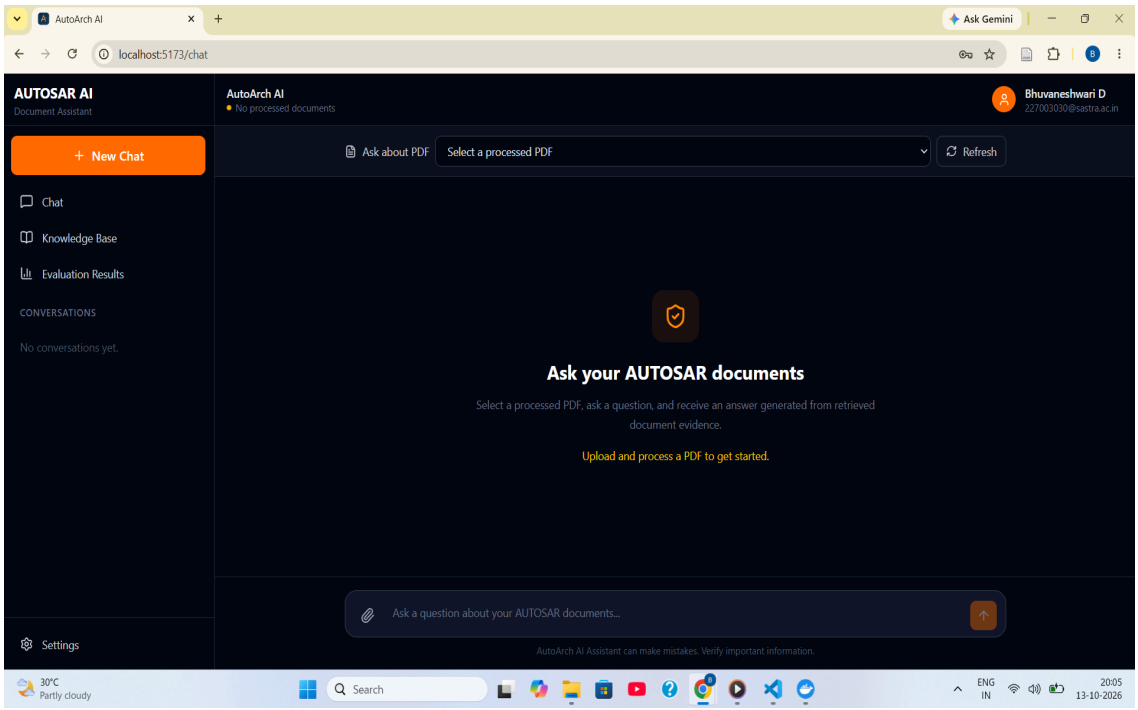


Figure 11.2: Application home page and user interface

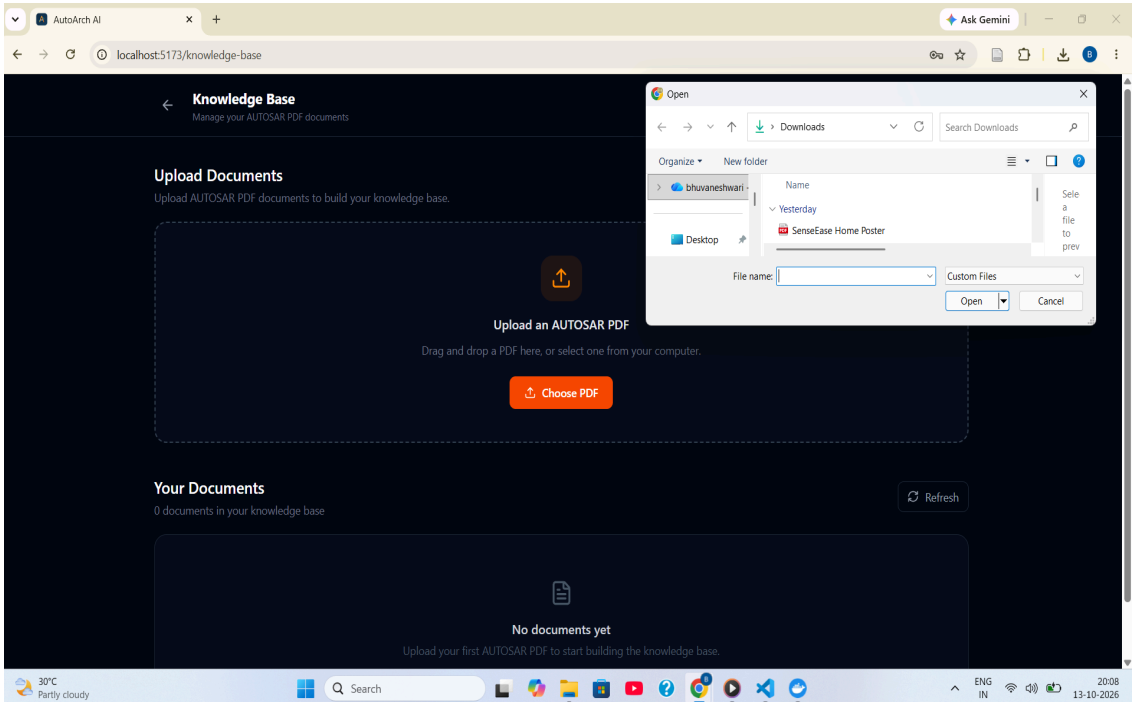


Figure 11.3: PDF document upload interface

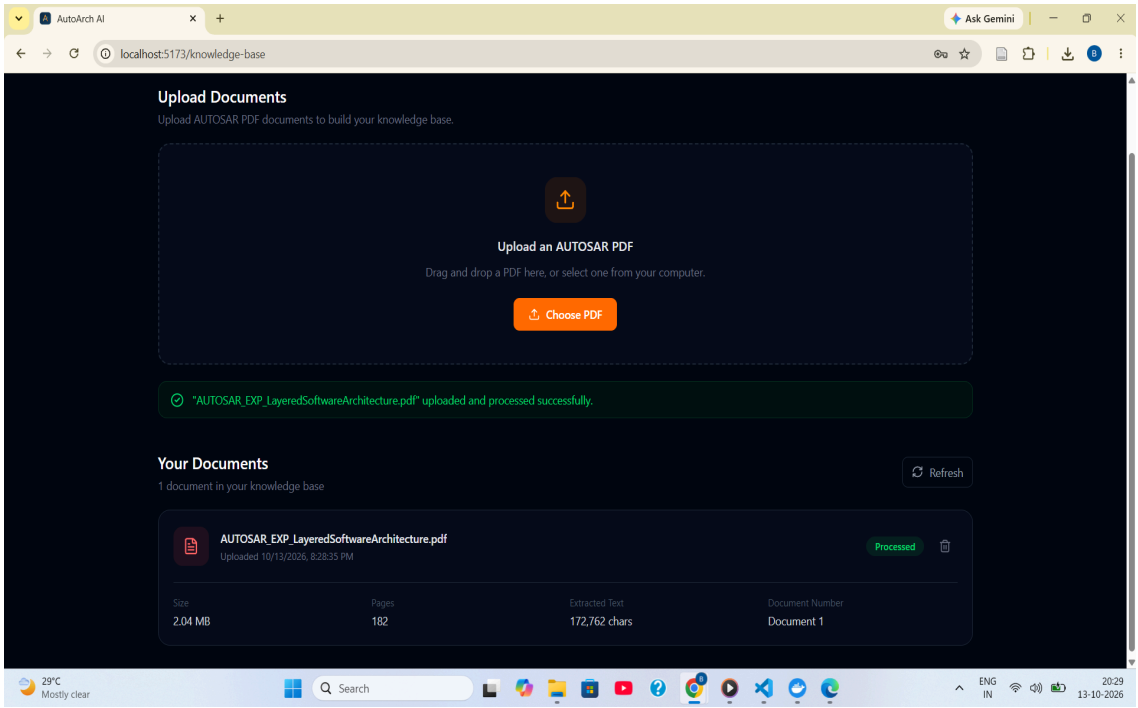


Figure 11.4: Uploaded document list and processing status

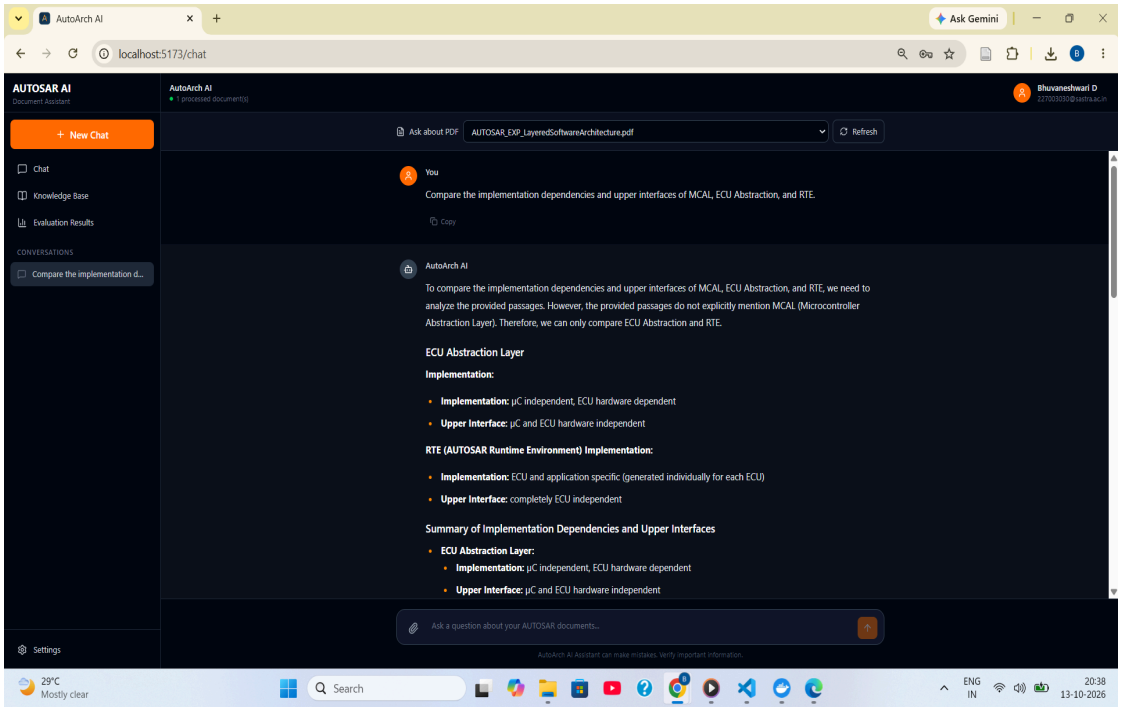


Figure 11.5: Question submitted about AUTOSAR HLD documentation

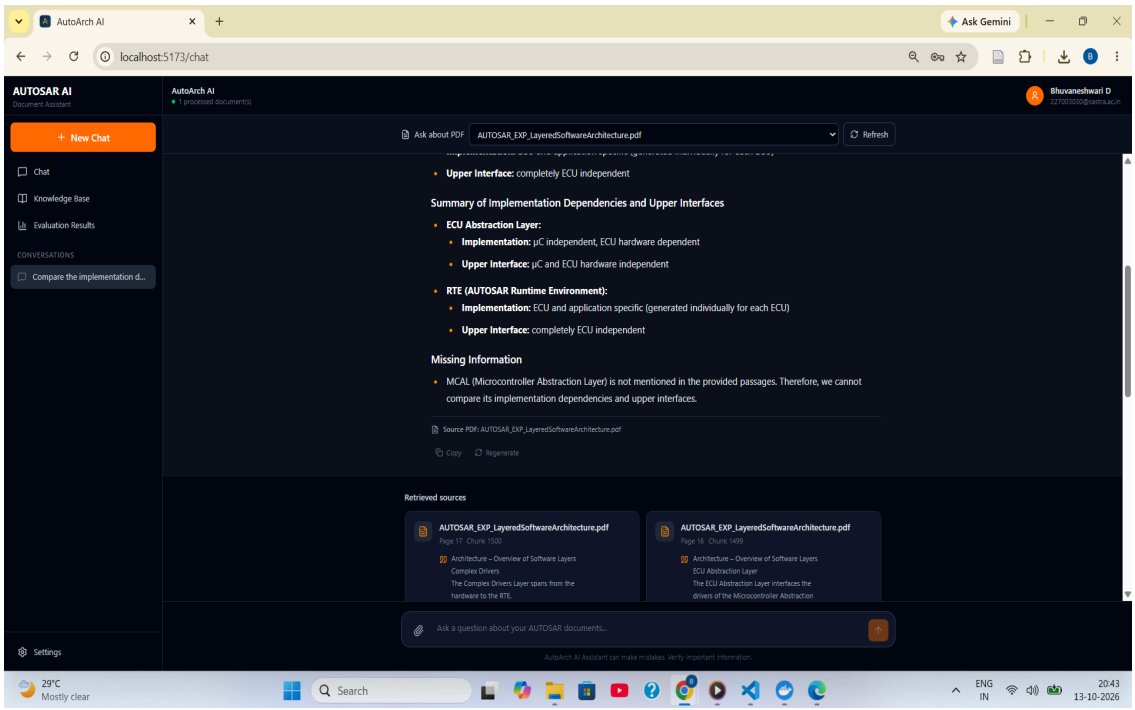


Figure 11.6: Generated answer with supporting document references

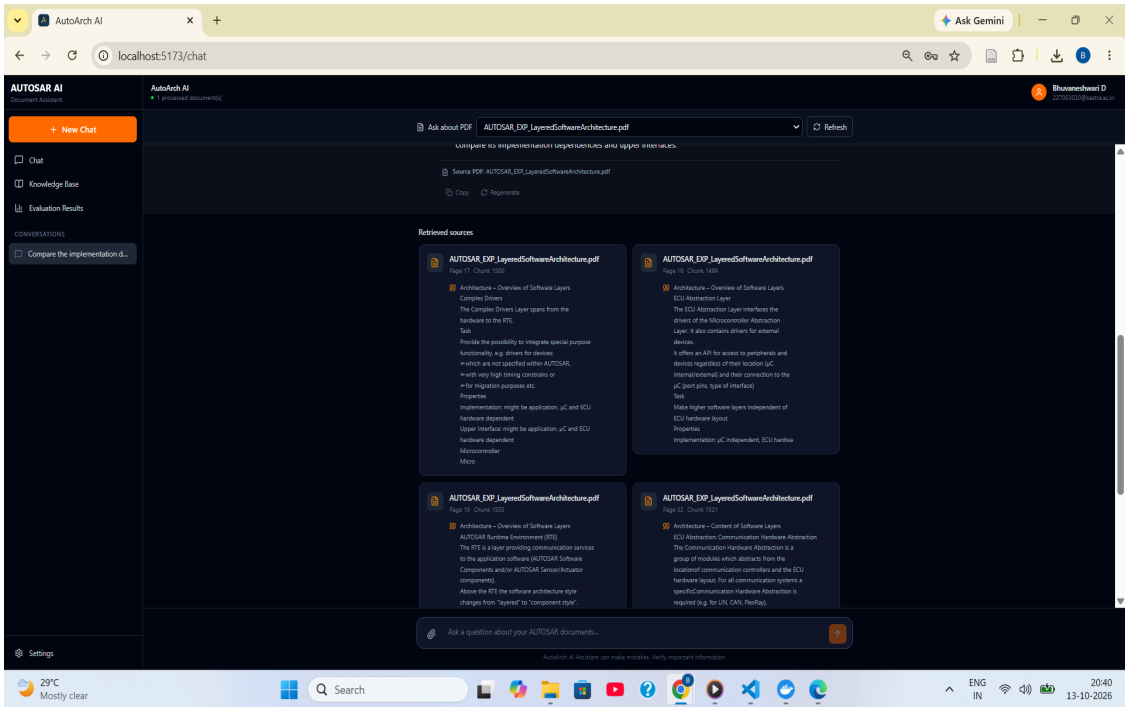


Figure 11.7: Source information used to verify an answer

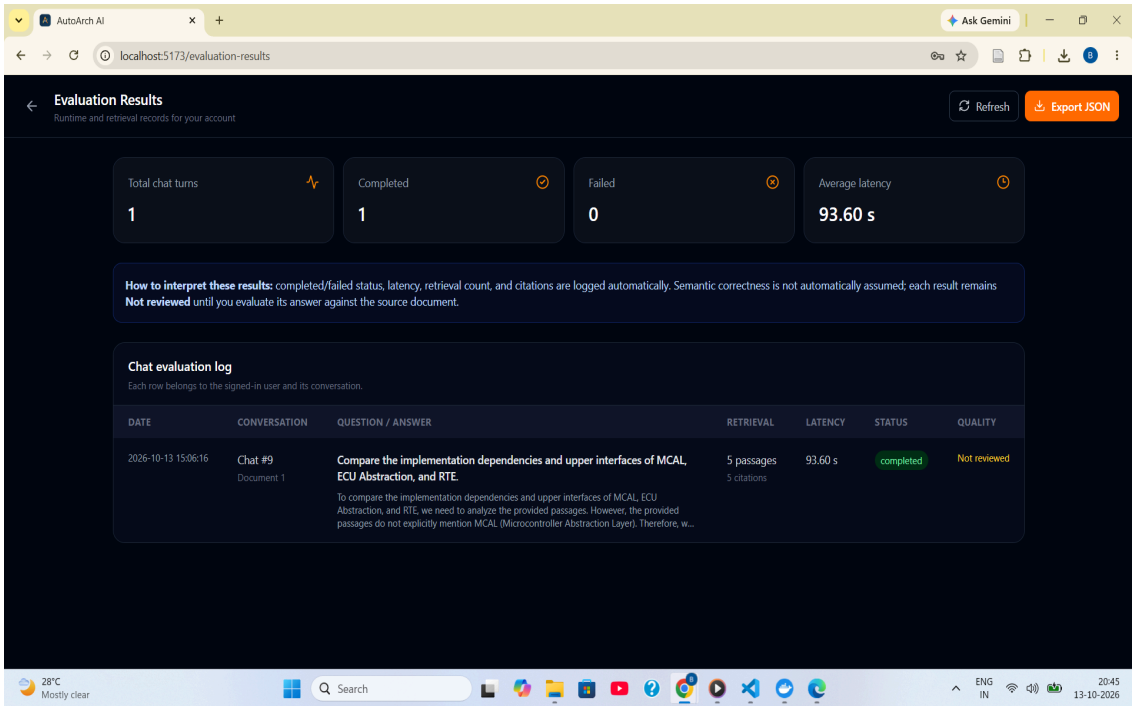

AutoArch AI
Ask Gemini
localhost:5173/evaluation-results
Evaluation Results
Runtime and retrieval records for your account
Refresh
Export JSON
Total chat turns
1
Completed
1
Failed
0
Average latency
93.60 s
How to interpret these results: completed/failed status, latency, retrieval count, and citations are logged automatically. Semantic correctness is not automatically assumed; each result remains Not reviewed until you evaluate its answer against the source document.
Chat evaluation log
Each row belongs to the signed-in user and its conversation.
DATE
CONVERSATION
QUESTION / ANSWER
RETRIEVAL
LATENCY
STATUS
QUALITY
2026-10-13 15:06:16
Chat #9
Document 1
Compare the implementation dependencies and upper interfaces of MCAL, ECU Abstraction, and RTE.
To compare the implementation dependencies and upper interfaces of MCAL, ECU Abstraction, and RTE, we need to analyze the provided passages. However, the provided passages do not explicitly mention MCAL (Microcontroller Abstraction Layer). Therefore, w...
5 passages
5 citations
93.60 s
completed
Not reviewed
28°C
Mostly clear
Search
ENG
IN
20:45
13-10-2026

Figure 11.8: Evaluation results page

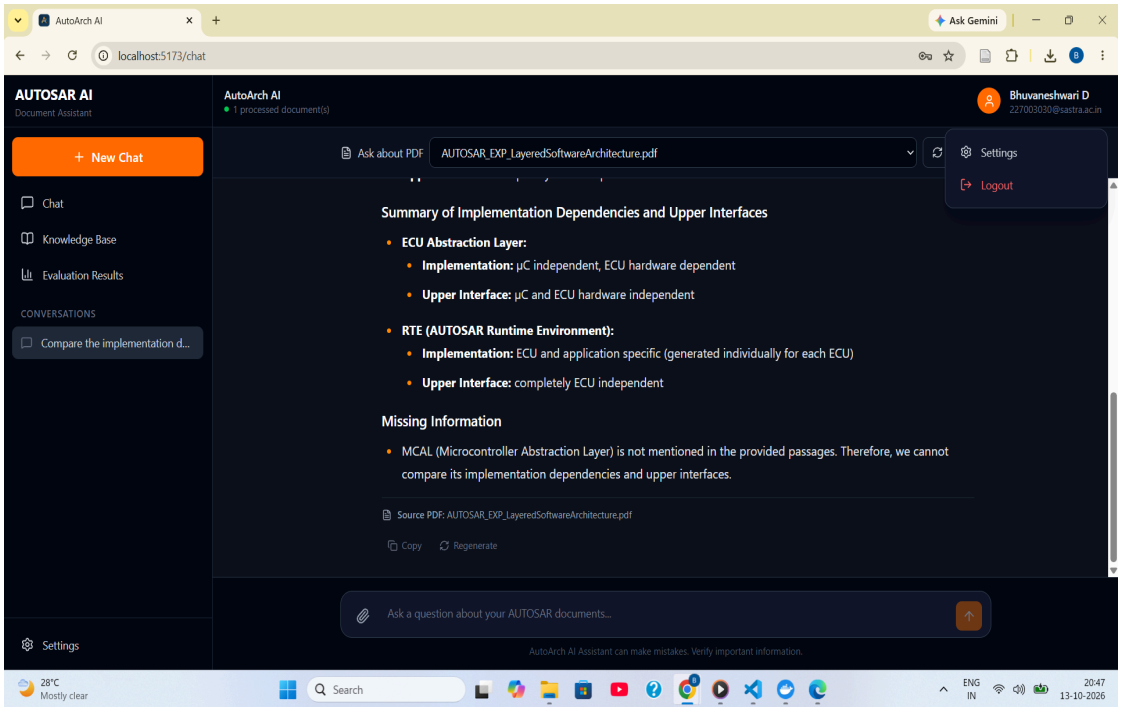


Figure 11.9: Dropdown options in user profile

Appendix B: Synopsis

Refer to the document attached at the end of this file.

Appendix C: Academic Integrity Declaration

Refer to the document attached at the end of this file.

Appendix D: AI-Tools Usage Declaration

Refer to the document attached at the end of this file.

SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act,1956)
THINK MERIT | THINK TRANSPARENCY | THINK SASTRA
THANJAVUR | KUMBAKONAM | CHENNAI

PROJECT SYNOPSIS

AI-ENABLED AUTOSAR HLD DOCUMENT ANALYSIS ASSISTANT

Student Name: Bhuvaneshwari D
Register Number: 227003030
Degree / Branch: B.Tech Computer Science and Engineering
University:: SASTRA Deemed University

ABSTRACT

AUTOSAR High-Level Design (HLD) documents have valuable information related to the components, interfaces, ports, signals, dependencies, and functional interactions in automotive software systems. Manually analyzing these documents can take up time, especially when architectural information is scattered across long technical documents. Finding the information and making sense of the relationship between the architecture elements could further increase the time needed for review and integration of the design. For addressing these issues, we present the AI-Powered AUTOSAR HLD Document Analysis Assistant that allows users to upload PDF documents and query any information present in these documents using natural language.The solution uses the Retrieval-Augmented Generation (RAG) framework for generating document-grounded responses. The frontend of the application is developed using React and Vite, while the backend is built using Python and Flask for handling document uploads, processing requests, and answering questions on the documents. PyMuPDF library is used for extracting textual information from the uploaded PDF documents. Text extracted from the documents is split into small chunks and represented as vector embeddings using the MiniLM embedding model. The vector embeddings are indexed using Facebook AI Similarity Search (FAISS).The documents are used as background information to the Qwen2.5 3B language model to generate natural language responses for query questions. The meta-information about the documents is stored in a SQLite database while the uploaded PDF documents, extracted texts and index information are stored in the filesystem.The developed assistant is designed to reduce the effort spent on documents search, access the architectural documents, and assist the engineers in design analysis and information retrieval.

Signature: D. Bhuvaneshwari

Date: 14/10/2026

Place: Kumbakonam

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act,1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

Academic Integrity Declaration

Project Title: AI-Enabled AUTOSAR HLD Document Analysis Assistant

Student Name: bhuvaneshwari D
Register Number: 227003030
University: SASTRA Deemed University
Degree: B.Tech Computer Science and Engineering

Declaration

I hereby declare that the capstone project entitled "**AI-Enabled AUTOSAR HLD Document Analysis Assistant**" is submitted as part of my academic project work.

I confirm that:

1. The project submission represents my original work, and all contributions made by me are accurately stated.
2. All external source code, datasets, pretrained models, documents, and other resources used in the project are appropriately acknowledged and cited.
3. The implementation, testing, evaluation, and results presented in the project documentation accurately reflect the work performed.
4. No evaluation results or experimental findings have been fabricated or deliberately misrepresented.

I understand the implementation and functionality of the submitted project and accept responsibility for the correctness and authenticity of my work.

Any assistance received during the development of this project has been appropriately acknowledged wherever required and I am responsible for maintaining academic integrity throughout the preparation, implementation, evaluation, and submission of this project.

Signature: D. Bhuvaneshwari

Date: 14/10/2026

Place: Kumbakonam

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

AI-TOOL USAGE DECLARATION

Project Title: AI-Enabled AUTOSAR HLD Document Analysis Assistant

Student Name: bhuvaneshwari D

Register Number: 227003030

University: SASTRA Deemed University

Degree: B.Tech Computer Science and Engineering

Declaration

I hereby declare that I have disclosed the AI tools and pretrained models actually used in the development of my capstone project, including their purposes and the project artefacts they affected. I have reviewed and verified the outputs used in my submission and accept responsibility for the correctness and integrity of my work. I understand that I must be able to explain the implementation and results of my project.

The following table records the AI tools and pretrained models used. I will ensure that each entry accurately reflects my actual use and that the verification method stated is one I performed.

AI Model/Tool	Purpose	Artifact affected	Verified by student
Qwen 2.5:3B via Ollama	Generate answers from retrieved AUTOSAR evidence	RAG answer-generation component	Yes
all-MiniLM-L6-v2	Generate text embeddings	Embedding and retrieval pipeline	Yes

Signature: D. Bhuvaneshwari

Date: 14/10/2026

Place: Kumbakonam

.